
Technical contract, proofs, and measurement conventions

Model and API contract

The reference code targets Python 3.10 or newer and requires no third-party package. Graphs are explicit adjacency lists with string node IDs, no parallel arcs, finite nonnegative int/float costs, and explicit entries for sinks. The graph and the heuristic remain fixed during one search. A state must contain all information relevant to future feasibility and cost; turn penalties or fuel constraints can require an augmented state. [R1, R4]

All classroom examples use small integer costs, so the displayed sums are exact. Floating-point inputs are accepted, but the code is not an arbitrary-precision numerical solver; extremely large floating sums can overflow. The stale check compares a queued sum with the exact value stored when that route was accepted. It does not use an epsilon to merge distinct paths.

A* accepts a mapping or callable. Values are cached on demand, checked for finiteness and nonnegativity, and $h(\text{target})=0$ is required. These checks do not establish admissibility. The caller must justify $h(v) \leq h^*(v)$ to claim optimality. There is no permanent closed set in the reference A*; its expanded set is used only to count re-expansions.

`check=False` skips graph validation for prevalidated benchmark inputs. It is not permission to run Dijkstra on negative edges. Deliberately incorrect variants are isolated in `algorithms/experiments.py`.

Dijkstra's finalization proof

Write $\text{delta}(v)$ for the true shortest-path cost from s . Let S be the finalized set. Induction hypothesis: $\text{dist}(x)=\text{delta}(x)$ for every x in S .

Let u be the non-stale minimum-distance node extracted from the frontier. Suppose $\text{dist}(u) > \text{delta}(u)$. On a shortest route to u , let y be the first node outside S and x its predecessor. When x was expanded, relaxation established $\text{dist}(y) \leq \text{delta}(x) + c(x,y) = \text{delta}(y)$. Every finite tentative label is the cost of a real route, so $\text{dist}(y) \geq \text{delta}(y)$, giving equality. Because the suffix from y to u has nonnegative cost, $\text{delta}(y) \leq \text{delta}(u)$. But u was the minimum candidate, so $\text{dist}(u) \leq \text{dist}(y) \leq \text{delta}(u)$, a contradiction. The first extraction s is the base case. [R1, R4]

The proof depends on nonnegative suffix cost, not on the choice of binary heap. A heap implements the required selection rule efficiently. Discovery of a target is not finalization. A non-stale target extraction justifies single-target termination; continuing until the queue empties computes all reachable distances.

A* with admissibility and reopening

Fix an optimal route with cost C^* . Until an optimal goal route is extracted, there is an appropriate frontier representative on that route with its optimal prefix cost, unless the corresponding prefix has already been expanded at that cost and propagated to its successor. Reopening ensures that an earlier, worse expansion cannot suppress this propagation. For such a representative v , $g(v) + h(v) \leq g^*(v) + h^*(v) = C^*$. A suboptimal goal has $f = g > C^*$ because $h(t)=0$ and cannot win minimum- f extraction while that representative exists. [R3; the reopening step is explicit here]

Do not claim that every current f -value is a lower bound on the globally optimal solution: the current g -value can already be a suboptimal prefix. The argument needs the appropriate optimal-prefix representative.

For this finite nonnegative graph model with strict improvements and a static finite heuristic, cycles do not create endlessly improving labels. An improved prefix can need to be propagated again under inconsistency. A* need not finalize every expanded state's g-value before termination.

Consistency and reweighting

Consistency requires $h(u) \leq c(u,v) + h(v)$ on every directed edge, with $h(t)=0$. Define

$$c_h(u,v) = c(u,v) + h(v) - h(u).$$

All reweighted edges are nonnegative exactly when h is consistent. Along a path from s to v the heuristic terms telescope:

$$g_h(v) = g(v) + h(v) - h(s) = f(v) - h(s).$$

Thus A* ordering is Dijkstra ordering in the reweighted graph, up to a constant. All s -to- t paths change cost by the same constant, so the optimal path is preserved. This supplies a finalization proof without reopening when h is consistent. Consistency also implies admissibility for nodes that can reach t , by telescoping along any route to t . [R4]

In the four-node example, reweighting turns $S \rightarrow B$ from 1 into 5 and $B \rightarrow A$ from 1 into -3. The failure to finalize safely is the earlier nonnegative-edge problem in another form.

Complexity for the delivered variants

Let $n=|V|$ and $m=|E|$. Exclude graph storage, visualization traces, and benchmark output from auxiliary memory. With $O(1)$ heuristic evaluation (or a cached heuristic with this cost per state), the bounds are:

Variant	Worst-case search time	Auxiliary memory
Simple-path enumeration	Number of routes can grow factorially	$O(n)$, excluding any optional stored outputs
Lazy Dijkstra, binary heap	$O(n + m \log(m+1))$	$O(n+m)$
Eager Dijkstra, indexed binary heap	$O((n+m) \log(n+1))$	$O(n)$
Lazy A*, consistent h	$O(n + m \log(m+1))$	$O(n+m)$

Lazy Dijkstra accepts at most one successful strict improvement per processed edge and can queue $O(m)$ records. Eager Dijkstra holds at most one live record per frontier node, and its mapping supports logarithmic decrease-key. On simple graphs $\log m = O(\log n)$, so these two binary-heap variants need not have different asymptotic time bounds. [R2, R5; direct analysis of the shipped code]

For A* with admissible but inconsistent h , edges can be scanned repeatedly after reopening; the one-expansion-per-node count does not apply. An explicit graph-size bound and an implicit search-space bound in branching factor and depth measure different quantities. A polynomial dependence on graph size does not imply a small implicit state space.

Fibonacci-heap Dijkstra has the classical amortized $O(m+n \log n)$ bound. This is not presented as the universal current state of the art for SSSP. A 2025 result provides $O(m \log^{(2/3)} n)$ for directed nonnegative real-weight graphs in a specified comparison-addition model. [R8] The lecture does not implement that algorithm or claim that its theoretical advantage implies faster classroom code.

Counter definitions

Expansion: processing a node's outgoing adjacency list, including repeated processing. The terminal goal extraction is not an expansion.

Re-expansion: any expansion after the first for the same state. These are counted only by A*; Dijkstra has none under its contract. Brute-force expansions count recursive prefixes whose outgoing arcs are inspected, not distinct vertices, so this is not a one-to-one state count.

Pop: every queue removal, including obsolete records and the goal. **Stale pop:** a record whose queued g no longer equals the best known g. **Push:** initial insertion or insertion of a new queue record; eager decreases are counted separately. **Peak queue:** maximum live array length, including obsolete records in lazy heaps. **Relaxation:** an outgoing edge inspected, even if it does not improve a label. **Improvement:** strict decrease in a tentative distance.

Brute force separately records **prefixes** and **complete paths examined**. It does not claim a queue size of zero makes it memory-free; it has a recursion stack. Its $O(n)$ memory refers to the reference in-place backtracking version, not the compact notebook's copied path/set convenience version.

Benchmark protocol and limitations

Fixtures are deterministic. Graph validation runs once outside timing. One warmup precedes each set of repeated timed calls. Time includes initialization, counter updates, heuristic evaluations, search, and path reconstruction; it excludes trace recording, rendering, graph generation, printing, and oracle checks. Report the median and minimum in milliseconds together with the interpreter and platform metadata.

The indexed heap is written in Python while `heapq` uses the interpreter's standard heap implementation. This is an educational implementation comparison, not a controlled language-neutral comparison of abstract data structures. Eager decrease-key need not win in practice. [R7]

Brute force has both a graph-size limit and a deterministic prefix budget. `budget_exhausted` means that even a returned incumbent is not certified optimal. Skipped runs receive no invented timing. The test suite independently checks small graphs against Bellman-Ford; the benchmark oracle is the already-tested Dijkstra implementation.

Visualizer provenance

`tools/trace_search.py` intercepts the actual Python function's line events. It records state after the previously executed line, excludes helper internals, and stores exact source lines with their origin. The browser replays these events. It never silently substitutes another search implementation.

The priority-order table is a sorted presentation; the heap-array table shows actual storage indices. Stale records are marked separately from re-expansions. Source highlighting means the line has executed, not that it is about to execute. A line that changes a distance can precede the next line that changes the predecessor; this transient program state is intentional.

References are resolved in `docs/references.md`.